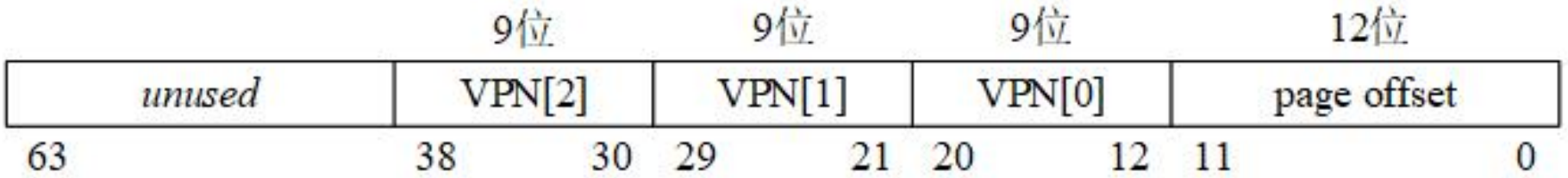
基于RISC-V代理内核的操作系统课程实验与课程设计

第四章．实验2：内存管理

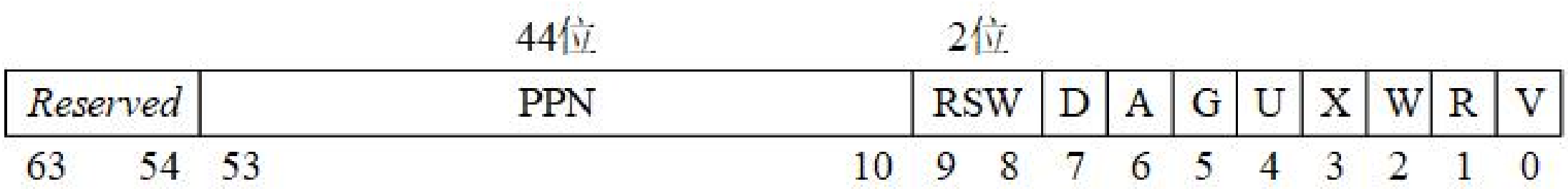
目录

实验2的基础知识

Sv39虚地址管理方案回顾

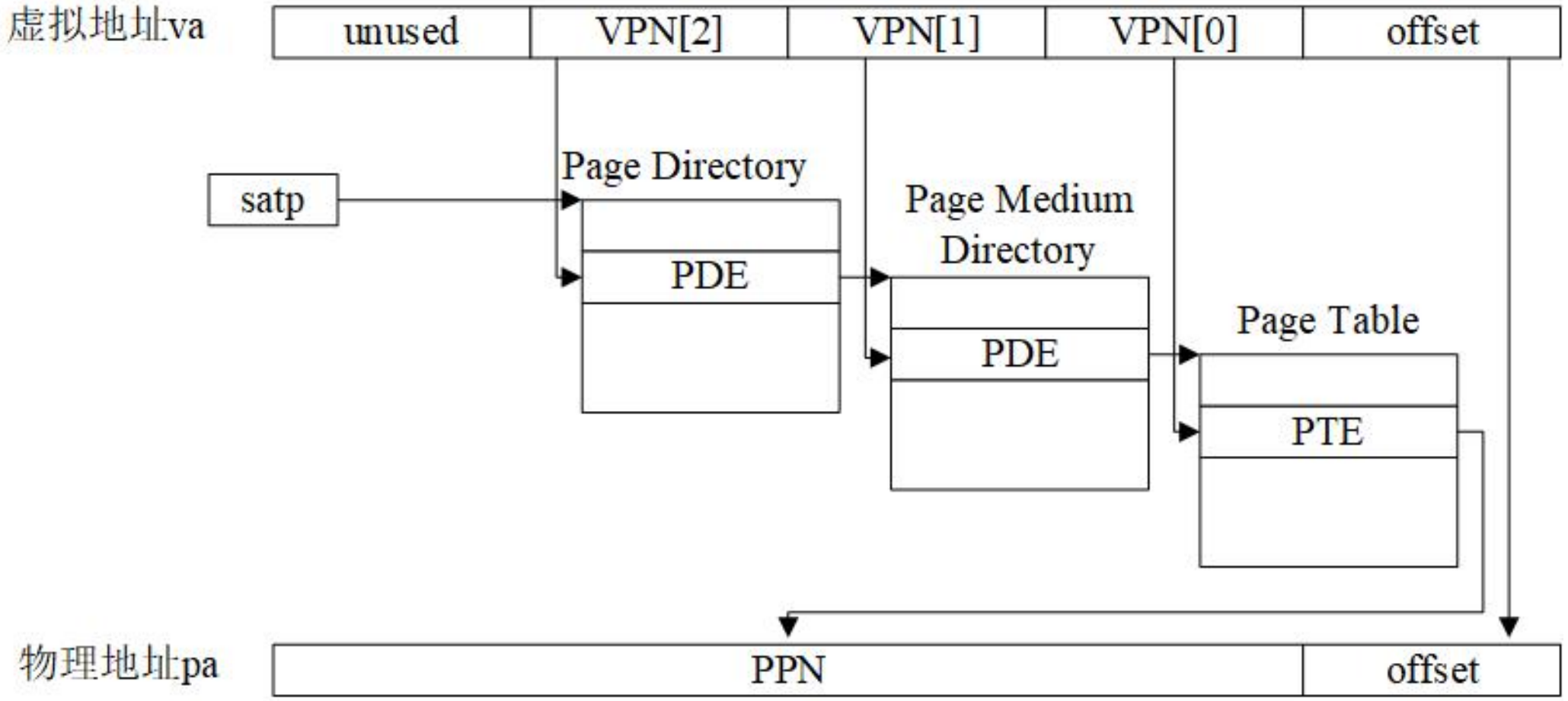


Sv39中逻辑地址的结构



Sv39中PDE/PTE格式

Sv39虚地址管理方案回顾



Sv39中虚拟地址到物理地址的转换过程

Sv39虚地址管理方案回顾

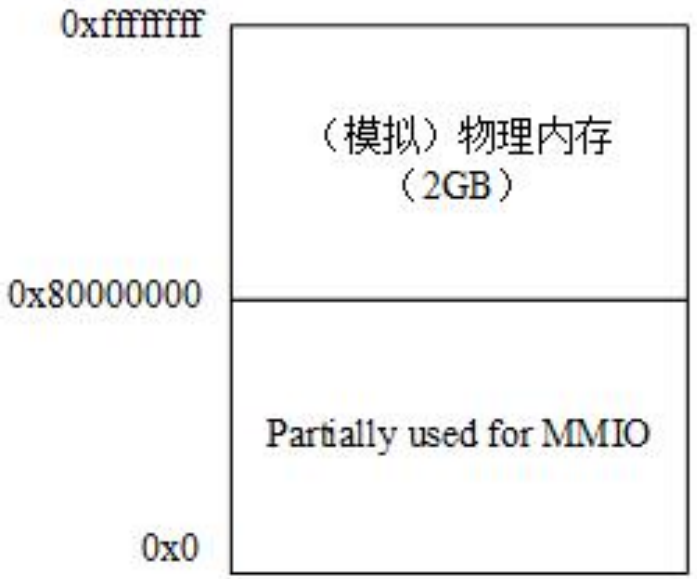
satp寄存器格式

satp寄存器中MODE域的取值和含义

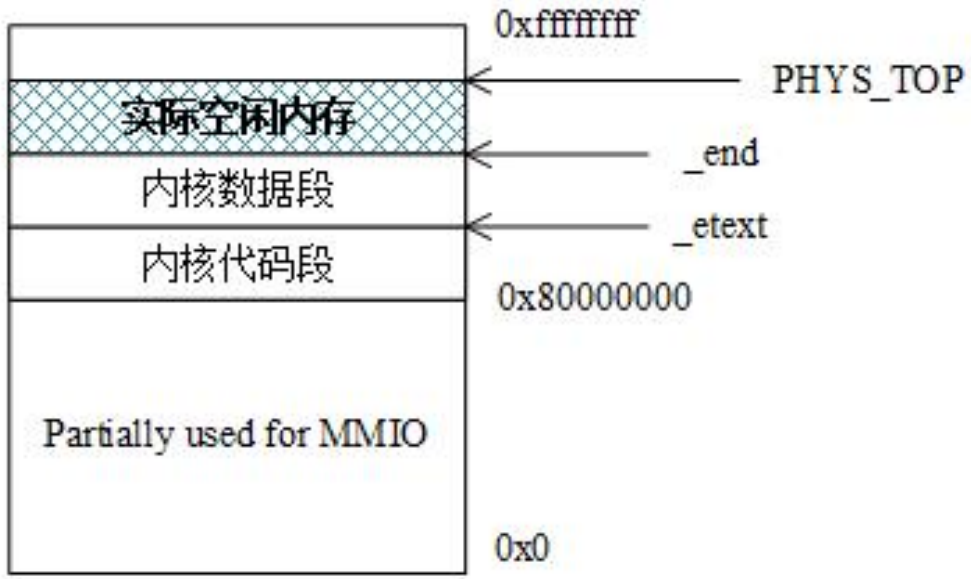
取值	虚存方案
0	Bare
8	Sv39
9	Sv48

物理内存布局与规划

对于我们用spike的模拟RISC-V机器而言，2GB的物理内存并不是从0地址开始编址，而是从0x80000000开始编址的。



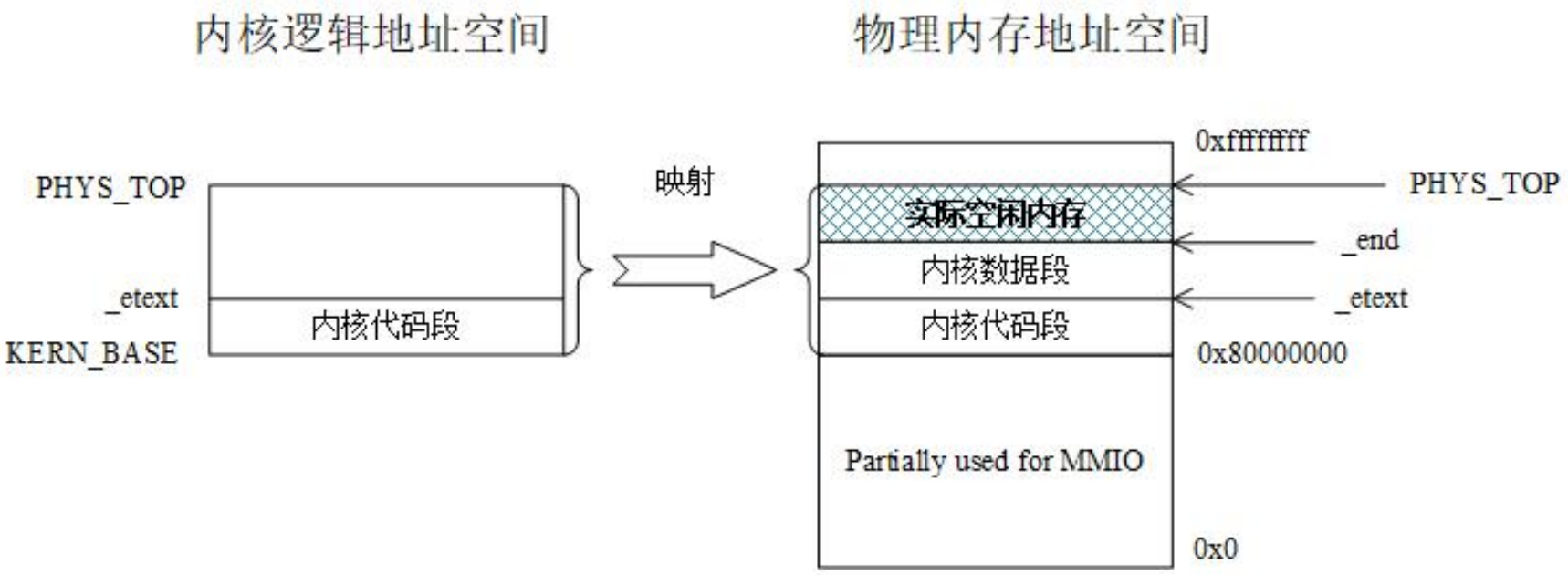
（a）RISC-V机器的初始内存布局



（b）装入PKE操作系统内核后的内存布局

PKE操作系统的逻辑地址空间结构

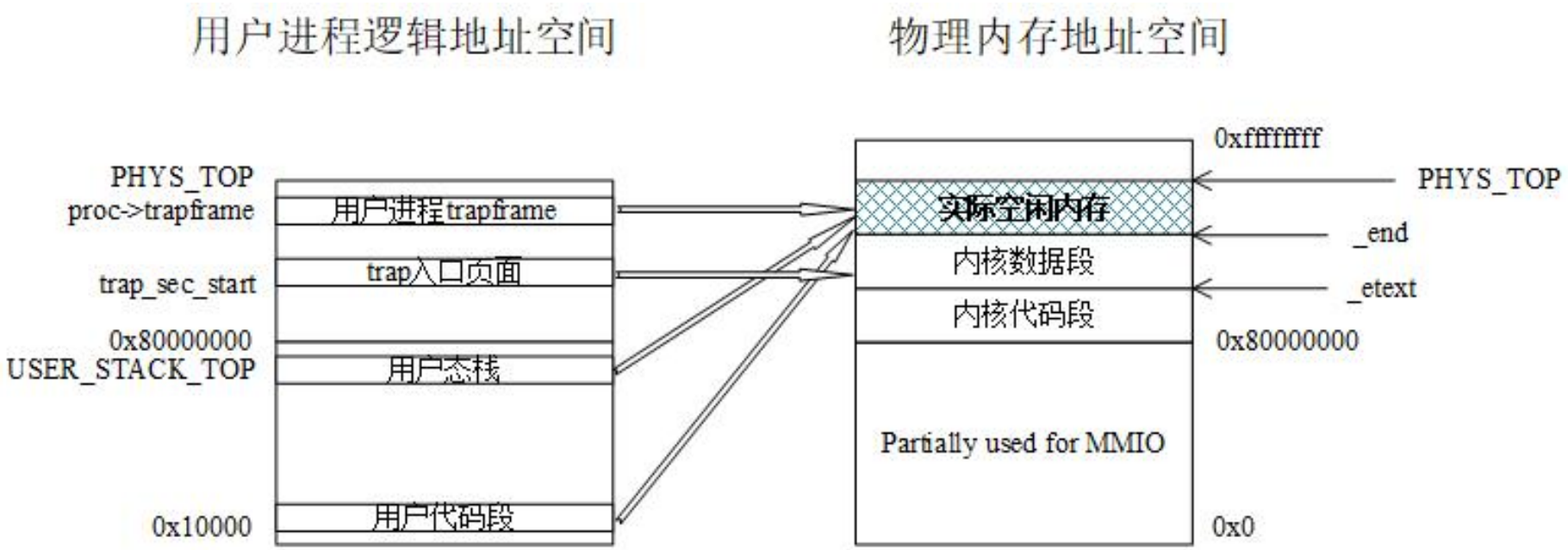
在开启了Sv39虚拟内存管理方案后，所有的逻辑地址到物理地址的翻译都必须通过页表和MMU硬件进行。



PKE操作系统内核的逻辑地址空间和它到物理地址空间的映射图

PKE应用进程的逻辑地址空间结构

lab2_1的应用app_helloworld_no_lds进程在装入后，其逻辑地址空间有4个区间建立了和物理地址空间的映射。



用户进程的逻辑地址空间到物理地址空间的映射图

与页表操作相关的重要函数

1、将逻辑地址映射到物理地址

- int map_pages(pagetable_t page_dir, uint64 va, uint64 size, uint64 pa, int perm);

2、查找逻辑地址所在的页表项

- pte_t *page_walk(pagetable_t page_dir, uint64 va, int alloc);

3、查找逻辑地址所在虚拟页面地址对应的物理页面地址

- uint64 lookup_pa(pagetable_t pagetable, uint64 va);

目录

- 实验2的基础知识
  - Sv39虚地址管理方案回顾
  - 物理内存布局与规划
  - PKE操作系统和应用进程的逻辑地址空间结构
  - 与页表操作相关的重要函数
- 实验内容
  - lab2_1 虚实地址转换
  - lab2_2 简单内存分配和回收
  - lab2_3 缺页异常

lab2_1 虚实地址转换

- user/app_helloworld_no_lds.c

```c
/*
 * Below is the given application for lab2_1.
 * This app runs in its own address space, in contrast with in direct mapping.
 */

#include "user_lib.h"
#include "util/types.h"

int main(void) {
  printu("Hello world!\n");
  exit(0);
}
```

给定应用

```
$ spike ./obj/riscv-pke ./obj/app_helloworld_no_lds
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x000000008000e000, PKE kernel size: 0x000000000000e000 .
free physical memory address: [0x000000008000e000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080004000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_helloworld_no_lds
Application program entry point (virtual address): 0x00000000000100f6
Switching to user mode...
Hello world!
User exit with code:0.
System is shutting down with exit code 0.
```

预期输出

lab2_1 虚实地址转换

实验内容:

开启Sv39页式地址管理，将应用（app_helloworld_no_lds.c，链接时未指定逻辑地址）投入正常运行。

实现user_va_to_pa()函数，完成给定逻辑地址到物理地址的转换，最终使得hello world程序获得正确输出。

lab2_2 简单内存分配和回收

- user/app_naive_malloc.c

```c
/*
 * Below is the given application for lab2_2.
 */

#include "user_lib.h"
#include "util/types.h"

struct my_structure {
  char c;
  int n;
};

int main(void) {
  struct my_structure* s = (struct my_structure*)naive_malloc();
  s->c = 'a';
  s->n = 1;

  printu("s: %lx, {%c %d}\n", s, s->c, s->n);

  naive_free(s);

  exit(0);
}
```

给定应用

```
$ spike ./obj/riscv-pke ./obj/app_naive_malloc
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x000000008000e000, PKE kernel size: 0x000000000000e000 .
free physical memory address: [0x000000008000e000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080004000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_naive_malloc
Application program entry point (virtual address): 0x0000000000010078
Switching to user mode...
s: 0000000000400000, {a 1}
User exit with code:0.
System is shutting down with exit code 0.
```

预期输出

lab2_2 简单内存分配和回收

实验内容:

这里，新定义了两个用户态函数naive_malloc()和naive_free()，它们最终会转换成系统调用，完成内存的分配和回收操作。

需要完成naive_free对应的功能，获得预期的输出。

lab2_3 缺页异常

- user/app_sum_sequence.c

```c
/*
 * The application of lab2_3.
 */

#include "user_lib.h"
#include "util/types.h"

//
// compute the summation of an arithmetic sequence. for a given "n", compute
// result = n + (n-1) + (n-2) + ... + 0
// sum_sequence() calls itself recursively till 0. The recursive call, however,
// may consume more memory (from stack) than a physical 4KB page, leading to a page fault.
// PKE kernel needs to improved to handle such page fault by expanding the stack.
//
uint64 sum_sequence(uint64 n) {
  if (n == 0)
    return 0;
  else
    return sum_sequence( n-1 ) + n;
}

int main(void) {
  // we need a large enough "n" to trigger pagefaults in the user stack
  uint64 n = 1000;

  printu("Summation of an arithmetic sequence from 0 to %ld is: %ld \n", n, sum_sequence(1000) );
  exit(0);
}
```

给定应用

```
$ spike ./obj/riscv-pke ./obj/app_sum_sequence
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x000000008000e000, PKE kernel size: 0x000000000000e000 .
free physical memory address: [0x000000008000e000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080004000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_sum_sequence
Application program entry point (virtual address): 0x0000000000010096
Switching to user mode...
handle_page_fault: 000000007fffdff8
handle_page_fault: 000000007fffcff8
handle_page_fault: 000000007fffbff8
Summation of an arithmetic sequence from 0 to 1000 is: 500500
User exit with code:0.
System is shutting down with exit code 0.
```

预期输出

lab2_3 缺页异常

实验内容:

应用程序执行时，由于采用递归函数求等差数列的和，递归层数过多，使得用户态栈溢出。

本实验中，我们处理的是缺页异常。

- 首先，判断我们处理的确实是缺页异常;
- 判断发生缺页的是不是用户栈空间，如果是则分配一个物理页空间，最后将该空间通过vm_map“粘”到用户栈上以扩充用户栈空间。